

DVR Optimization Problem

Jason Joy Manoj

July 26, 2026

1 Notation

- v : the verifier assigned to check the draft model’s output. Takes one of four values: none, small, medium, large.
- k : the draft length, the number of tokens the draft model proposes before verification. Takes one of three values: 3, 5, or 7. Only applies when $v \neq \text{none}$.
- d : the draft model used to propose tokens for a query. Takes one of $|D|$ values.
- D : the set of all available draft models on Node 1.
- b : the batch size at which a candidate was profiled offline (e.g. $b \in \{4, 8, 16\}$). Distinct from n , the size of a live incoming batch B .
- (d, v, k) : a candidate, one specific combination of draft model, verifier, and draft length. For each draft model $d \in D$, there are 10 candidates: none (1), plus small, medium, large each at $k \in \{3, 5, 7\}$ (9). This gives $10|D|$ candidates total per query.
- query: the text of the incoming prompt.
- $B = \{\text{query}_1, \dots, \text{query}_n\}$: a batch of n queries arriving together for routing.
- i : index of a query within the current batch, $i \in \{1, \dots, n\}$.
- f_{enc} : a frozen, pretrained sentence encoder (all-mpnet-base-v2) that converts text into a fixed-length vector.
- $g(d, v, k)$: the candidate’s embedding, built from a written description of draft model d and verifier v , both passed through the same encoder, with the numeric value of k concatenated alongside them.
- $z_i(d, v, k) = f_{\text{enc}}(\text{query}_i) \oplus g(d, v, k)$: the combined input vector fed to the router MLP, for query i and candidate (d, v, k) .
- $\sigma(x) = \frac{1}{1 + e^{-x}}$: the sigmoid function.
- $A_i(d, v, k)$: the router’s predicted accuracy for candidate (d, v, k) , given query i in the current batch. Computed live, once per query, for all $10|D|$ candidates.
- $E(d, v, k, b)$: the raw measured energy cost (grams of CO₂) of candidate (d, v, k) when profiled at batch size b , measured offline via CodeCarbon.
- $T_{\text{raw}}(d, v, k, b)$: the raw measured latency (draft time plus verification time plus cross-node transmission time) of candidate (d, v, k) when profiled at batch size b , measured offline.
- $\text{EnergyNorm}(d, v, k, b)$, $\text{TimeNorm}(d, v, k, b)$: $E(d, v, k, b)$ and $T_{\text{raw}}(d, v, k, b)$ rescaled to $[0, 1]$ via min-max normalization across the $10|D|$ candidates profiled at the same batch size b . Higher value means worse (more carbon, slower).

- w_A, w_E, w_T : independent weights, each in $[0, 1]$, summing to 1, controlling how much Accuracy, Energy, and Time count toward the final decision.
- $\text{CombinedScore}(v, k)$: the final score computed for each candidate.
- (d_i^*, v_i^*, k_i^*) : the winning candidate for query i - the draft model, verifier, and draft length used to answer that specific query in the batch.

2 Accuracy Term

For each of the 10 candidates (v, k) , the predicted accuracy $A(v, k)$ is produced by a four-step calculation:

1. The query is encoded into a vector by f_{enc} . Separately, a written description of verifier v is encoded by the same encoder, and the numeric value of k is attached alongside it. These are concatenated into $z(v, k)$.
2. $z(v, k)$ is passed through the router MLP, producing one raw scalar.
3. The raw scalar is passed through σ , squashing it into $[0, 1]$.
4. The result is $A(v, k)$, the predicted accuracy for that candidate.

$$A_i(d, v, k) = \sigma(\text{MLP}(z_i(d, v, k))) \quad (1)$$

Description: $A(v, k)$ is the router’s live, per-query estimate of how likely candidate (v, k) is to answer this specific query correctly. It is the only one of the three objective terms that is computed on the fly for every incoming query; Energy and Time are looked up from a fixed offline table (Section 3).

This is repeated for every query i in the batch B , and for all $10|D|$ candidates per query, producing $10|D|$ accuracy values per query and $n \times 10|D|$ accuracy values per batch.

Training labels for $A(v, k)$ are gathered offline: the pipeline is run on datasets with known correct answers, trying different (v, k) combinations, and each run’s output is scored against the gold-standard answer to produce a label.

3 Energy and Time Terms

Unlike accuracy, the Energy and Time terms do not require a live predictor. Both are known, fixed values per candidate, measured once offline and reused for every query.

Since the per-query energy and latency cost of a candidate depends on how many queries it processes together in a single batch under vLLM’s batching and parallel verification, a single profiling table is not sufficient. Instead, a separate offline profiling table is built for each of a small set of representative batch sizes (e.g. $b \in \{4, 8, 16\}$). For a given batch size b , every one of the $10|D|$ candidates is run on a fixed offline profiling workload at that batch size, and $E(d, v, k, b)$ (energy) and $T_{\text{raw}}(d, v, k, b)$ (latency) are recorded across the profiling queries, then aggregated (e.g. averaged) into a single representative value per candidate. This produces one $10|D|$ -row table per batch size, each with a column of raw E values and a column of raw T_{raw} values. EnergyNorm and TimeNorm are computed by scanning down a given table’s E column (or T_{raw} column) to find the minimum and maximum entries *within that same batch size’s table*, then rescaling every row’s value relative to those two numbers. At runtime, once the Batch Scheduler assembles an actual live batch of size n , DVR looks up EnergyNorm and TimeNorm from whichever profiled table’s b most closely matches n . These tables are built once offline and reused, unchanged, for every incoming query.

$$\text{EnergyNorm}(d, v, k, b) = \frac{E(d, v, k, b) - \min_{d', v', k'} E(d', v', k', b)}{\max_{d', v', k'} E(d', v', k', b) - \min_{d', v', k'} E(d', v', k', b)} \quad (2)$$

Description: Rescales the raw measured CO₂ cost of each candidate onto $[0, 1]$, relative to the cheapest and most expensive candidates in the expanded pool of $10|D|$ candidates *profiled at the same batch size b* .

The candidate with the lowest measured energy (at that b) gets $\text{EnergyNorm} = 0$; the most expensive (at that b) gets $\text{EnergyNorm} = 1$. This is a min-max normalization, computed once offline per batch-size table and reused unchanged for every query whose live batch matches that b .

$$\text{TimeNorm}(d, v, k, b) = \frac{T_{\text{raw}}(d, v, k, b) - \min_{d', v', k'} T_{\text{raw}}(d', v', k', b)}{\max_{d', v', k'} T_{\text{raw}}(d', v', k', b) - \min_{d', v', k'} T_{\text{raw}}(d', v', k', b)} \quad (3)$$

Description: Same min-max normalization, applied to raw measured latency instead of energy, within the same batch-size table. Both Eq. 2 and Eq. 3 produce values in $[0, 1]$ where higher means worse (more carbon-costly, or slower) — oriented so that both are subtracted, not added, in the final score.

4 Combined Optimization Problem

We want to, for a given query:

$$\begin{aligned} &\text{maximize} && A_i(d, v, k) \\ &\text{minimize} && \text{EnergyNorm}(d, v, k, b) \\ &\text{minimize} && \text{TimeNorm}(d, v, k, b) \end{aligned}$$

These three goals are combined into a single score per candidate, with each objective given its own independent weight:

$$\text{CombinedScore}_i(d, v, k) = w_A \cdot A_i(d, v, k) - w_E \cdot \text{EnergyNorm}(d, v, k, b) - w_T \cdot \text{TimeNorm}(d, v, k, b) \quad (4)$$

Description: A weighted linear combination (scalarization) of the three objectives into a single number per candidate, computed independently for each query i in the batch. Accuracy is added because higher is better; Energy and Time are subtracted because higher is worse. EnergyNorm and TimeNorm are looked up from the offline profiling table whose batch size b most closely matches the current live batch size n — all queries within the same batch B share the same n , and therefore the same $\text{EnergyNorm}/\text{TimeNorm}$ table, though each query i still receives its own live-computed accuracy $A_i(d, v, k)$. The three weights are independent of one another, so each objective’s importance can be tuned separately during ablations (e.g. an energy-first configuration raises w_E without changing w_T).

The optimization problem is:

$$(d_i^*, v_i^*, k_i^*) = \arg \max_{(d, v, k)} \text{CombinedScore}_i(d, v, k) \quad \forall i \in \{1, \dots, n\} \quad (5)$$

$$\begin{aligned} \text{subject to: } & d \in D \\ & v \in \{\text{none}, \text{small}, \text{medium}, \text{large}\} \\ & k \in \{3, 5, 7\} \text{ if } v \neq \text{none} \\ & w_A + w_E + w_T = 1, \quad w_A, w_E, w_T \geq 0 \end{aligned}$$

Description: Out of the $10|D|$ candidates, the router selects the (d, v, k) triple with the highest CombinedScore for each query i ; this becomes the draft model, verifier, and draft length used for that query. This argmax is solved independently for every query in the batch B - each query receives its own (d_i^*, v_i^*, k_i^*) based purely on its own predicted accuracy, since Energy and Time are query-independent offline lookups. The weight-sum constraint is not required for the argmax itself to function (argmax is invariant to uniform rescaling) — it is imposed so that w_A, w_E, w_T remain interpretable as a trade-off allocation across the three objectives when swept during ablations. The resulting n routing decisions are then handed to the Batch Scheduler, which assigns them to physical nodes subject to node capacity constraints - a separate combinatorial assignment problem (JSQ-style load balancing) not encoded inside CombinedScore itself.

Note: The optimization problem itself does not summate or average Accuracy, Energy, or Time across the batch B . Eq. 5 is solved independently for each query i , producing n separate routing decisions with no batch-level aggregation feeding back into the decision. Batch-level totals or averages (e.g. total energy consumed by a batch, mean accuracy across a batch) may still be computed afterward for reporting and evaluation purposes, but this is a separate, after-the-fact summary statistic and plays no role in CombinedScore or the argmax.

5 Background: Why Latency Varies with k and Verifier Size

$T_{\text{raw}}(v, k, b)$ is measured directly rather than derived analytically, but the reason latency varies across candidates is well captured by the closed-form speedup expression from speculative decoding theory [1]. Given an acceptance rate α , a draft length γ , and a cost ratio c (time for one draft-model pass divided by time for one verifier-model pass), the expected speedup over standard autoregressive decoding is:

$$\text{Speedup}(\alpha, \gamma, c) = \frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)(\gamma c + 1)} \quad (6)$$

Description: A higher acceptance rate increases expected speedup, since more drafted tokens survive verification per call. A larger cost ratio (verifier slow relative to the drafter) reduces speedup, since $(\gamma c + 1)$ grows with $\gamma = k$. This motivates why larger verifiers and larger k trade accuracy against latency in opposite directions — but it is not used directly inside CombinedScore, since $T_{\text{raw}}(v, k, b)$ is taken from offline profiling rather than computed live per query.

- **Eq. 1 (Accuracy term structure). Sourced.** Directly adapted from RTR’s performance predictor, $\hat{a}_{i,j,k} = \sigma(\text{MLP}_{\text{perf}}(z_{i,j,k}))$ (RTR, WWW 2026, Eq. 4). Same structure, renamed to match DVR’s notation.
- **Eqs. 2–3 (Min-max normalization). Not sourced from a specific paper.** A standard, general-purpose normalization technique used broadly in statistics and ML; not tied to any single publication. Adopted here as the author’s own design choice to give Energy and Time comparable $[0, 1]$ scales, replacing an earlier undefined normalization step.
- **Eq. 4 (Weighted three-term CombinedScore). Not sourced from a specific paper.** A standard linear scalarization approach from multi-objective optimization. Structurally similar in spirit to RTR’s own two-term, single-weight score (RTR, Eq. 9), extended here to three independently-weighted terms. This extension is the author’s own formulation, not copied from a published equation.
- **Eq. 6 (Speedup formula). Sourced.** Verified directly against Leviathan, Kalman & Matias, *Fast Inference from Transformers via Speculative Decoding*, ICML 2023 (arXiv:2211.17192) [1]. Formula and definition of the cost ratio c both confirmed to match the original.

Summary: 2 of the 4 core equations are adapted directly from cited papers (Eq. 1 from RTR, Eq. 6 from Leviathan et al.); 2 are standard techniques applied as the author’s own design choices (Eqs. 2–3, 4), not attributable to a single source.

References

- [1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. *Fast Inference from Transformers via Speculative Decoding*. Proceedings of the 40th International Conference on Machine Learning, PMLR 202:19274–19286, 2023. arXiv:2211.17192.